

서블릿 위임과 세션, 쿠키, JSP

오늘의 강의 학습 요약

금일 수업은 서블릿의 라이프사이클(단일 인스턴스)로 인해 발생하는 스레드 안전성 이슈를 복습한 뒤, 서블릿이 화면(HTML) 생성에 취약하므로 화면 처리를 JSP로 위임해야 하는 이유를 정리했습니다. 이어서 요청 위임의 두 방식인 리다이렉트와 포워드를 API/실습 관점에서 비교하고, 각 방식에서 URL 변화 및 request scope 데이터 전달 가능 여부가 어떻게 달라지는지를 핵심 포인트로 다뤘습니다.

이후 HTTP의 Connectionless 특성 때문에 페이지(서블릿/JSP) 간 상태 공유가 불가능하다는 문제를 제시하고, 이를 해결하는 세션/쿠키 기반 세션 관리의 구조와 구현 패턴(저장/조회/검증/로그아웃)을 실습으로 정리했습니다. 마지막으로 JSP의 실행 단계와 태그 구성 요소, 그리고 모델1/모델2(MVC) 아키텍처 차이를 통해 “브라우저는 서블릿(컨트롤러)만 요청한다”는 모델2 규칙을 강조했습니다.

1. 서블릿 핵심 복습과 화면 분리

서블릿은 단 한 번 생성되는 단일 인스턴스 기반이므로 인스턴스 변수 사용 시 스레드-언세이프 위험이 있으며, 화면(HTML) 생성은 JSP 같은 뷰 컴포넌트로 위임하는 구조가 적합합니다.

서블릿은 기본적으로 한 번만 생성되어 재사용되는 구조이므로, 인스턴스 변수에 상태를 담으면 동시 요청에서 값이 섞일 수 있어 위험합니다. 따라서 요청 처리 중 필요한 값은 가급적 지역 변수(로컬 변수)로 다루어 스레드 세이프하게 구성해야 합니다.

서블릿의 `doGet/doPost` 는 항상 `HttpServletRequest` , `HttpServletResponse` 를 파라미터로 가지며, 요청 데이터 획득과 응답 처리를 담당합니다. 응답 처리는 본질적으로 브라우저가 렌더링할 HTML을 만들어 전달하는 과정이며, `Content-Type(MIME Type)` 설정 후 `response.getWriter()` 로 HTML 문자열을 출력하는 방식이 기본입니다.

서블릿은 비즈니스 로직 처리에는 강점이 있지만, 문자열로 HTML을 만드는 방식은 유지보수성과 생산성이 떨어집니다. 이 한계를 해결하기 위해 화면 생성은 HTML 중심 구조인 JSP에 맡기고, 서블릿은 로직 처리(서비스 연동, 파라미터 처리, 데이터 준비)에 집중하도록 역할을 분리합니다.

요청 처리에서는 HTML 폼 기반 입력 전달이 대표적이며, GET은 URL(Query String)에 포함되고 POST는 Body에 포함된다는 차이를 전제로 파라미터를 `request.getParameter(...)` 등으로 가져옵니다. 체크박스 처럼 동일 name에 다중 값이 올 수 있는 경우를 포함해, 파라미터 획득 메소드들의 사용 맥락을 구분해야 합니다.

또한 데이터 보관 범위인 스코프는 `request/session/application` 중심으로 이해해야 하며, `page scope` 는 페이지 내부 범위로 자바의 지역 범위와 유사해 실무적으로 비중이 낮습니다.

스코프는 "데이터를 얼마나 오래 유지해야 하는가" 관점으로 선택해야 하며, 불필요하게 긴 스코프에 큰 데이터를 올리면 메모리/성능 문제가 커집니다.

스코프	수명(라이프사이클)	대표 용도	비고
Request	요청~응답까지로 매우 짧습니다.	게시판 목록처럼 보여주고 끝나는 데이터에 적합합니다.	응답 완료 시 소멸됩니다.
Session	동일 브라우저가 유지되는 동안(타임아웃 전)입니다.	로그인 상태, 브라우저 종료 시 사라져도 되는 장바구니에 적합합니다.	타임아웃은 <code>web.xml</code> 또는 코드로 설정합니다.
Application	톰캣(컨테이너) 종료 전까지입니다.	방문자 수처럼 전체 사용자에게 공유되는 누적 성격에 적합합니다.	DB 미사용 가정에서 예시로 사용됩니다.

세션 타임아웃은 코드로도 설정할 수 있으며, 단위는 초입니다.

```
session.setMaxInactiveInterval(1800); // 30분(초 단위)
```

핵심 키워드

#서블릿 라이프사이클

#스레드 세이프

#HttpServletRequest

#HttpServletResponse

#스코프

2. 필터 개념과 맵핑, 순서

필터는 서블릿 앞/뒤에 끼워 요청, 응답을 가로채는 컴포넌트이며, `web.xml` 맵핑과 선언 순서에 따라 적용 대상과 실행 순서가 결정됩니다.

필터는 “서블릿이 최종 요청을 받기 전에” 공통 처리를 삽입하기 위한 구조입니다. 요청이 서블릿으로 들어가기 전 뿐 아니라, 응답이 브라우저로 나가기 전에도 개입할 수 있어 로깅, 인코딩 처리, 인증/권한 검사 같은 횡단 관심사를 구현할 때 유용합니다.

구현은 `Filter` 를 구현하고 `doFilter(ServletRequest, ServletResponse, FilterChain)` 를 재정의하는 형태이며, 요청/응답 타입이 `HttpServletRequest/Response` 가 아닌 상위 타입 (`ServletRequest/Response`)로 전달된다는 점을 함께 인지해야 합니다.

필터 맵핑은 모든 서블릿에 적용할지, 특정 서블릿(특정 URL 패턴)에만 적용할지 결정합니다. `/ *` 패턴은 “모든 서블릿 요청에 필터를 적용”한다는 의미이며, 특정 맵핑값을 주면 해당 서블릿 요청에만 필터가 적용됩니다.

필터를 여러 개 적용할 수 있으며, `web.xml`에서는 DTD/XSD 규칙상 선언 순서가 중요합니다. 즉, 먼저 선언된 필터가 먼저 실행되며, 순서를 뒤집으면 실행 순서도 바뀝니다.

필터 개념은 이후 스프링(특히 스프링 시큐리티) 이해에도 직접 연결됩니다. 스프링 시큐리티는 다수의 필터 체인으로 인증/인가를 구현하므로, “요청이 서블릿/컨트롤러에 도달하기 전에 필터로 검증한다”는 관점을 선행 이해하는 것이 중요합니다.

핵심 키워드

#Filter

#FilterChain

#web.xml 맵핑

/* 패턴

#스프링 시큐리티

3. 요청 위임: 리다이렉트와 포워드

요청 위임은 서블릿이 화면 컴포넌트(JSP 등)를 선택해 응답을 맡기는 과정이며, 리다이렉트는 재요청으로 URL과 request가 바뀌고 포워드는 request를 재사용해 URL이 유지됩니다.

요청 위임은 “서블릿이 JSP(또는 다른 서블릿/HTML)에게 응답 처리를 맡기기 위해 대상 컴포넌트를 선택하는 것”입니다. 핵심은 역할 분리이며, 서블릿은 비즈니스 로직과 데이터 준비를 담당하고, JSP는 화면 렌더링을 담당합니다.

요청 위임 방식은 크게 `Redirect` 와 `Forward` 두 가지이며, 둘 다 “요청을 통해 대상 리소스를 선택한다”는 점에서 브라우저가 URL로 요청하는 방식과 유사한 효과를 가집니다.

리다이렉트는 `HttpServletResponse.sendRedirect(String location)` 을 사용합니다. 실행 흐름상 서블릿이 브라우저에 “다시 이 URL로 요청하라”는 응답을 보내고, 브라우저가 실제로 새로운 요청을 생성하여 대상(JSP 등)으로 접근합니다.

```
response.sendRedirect("target.jsp");
```

리다이렉트의 특징은 URL이 변경된다는 점이며, 요청이 두 번 발생하므로 `HttpServletRequest` 도 두 개가 생성됩니다. 따라서 첫 요청에서 `request.setAttribute(...)` 로 저장한 데이터는 두 번째 요청의 request에는 존재하지 않아 전달되지 않습니다.

반면 포워드는 `request.getRequestDispatcher(path).forward(request, response)` 를 사용합니다. 서블릿이 브라우저로 응답하지 않고, 서버 내부에서 동일 요청/응답 객체를 유지한 채 대상 리소스에게 처리를 넘깁니다.

```
request.getRequestDispatcher("target2.jsp").forward(request, response);
```

포워드는 URL이 변경되지 않으며, 요청 객체가 재사용되므로 `request scope` 에 저장한 데이터를 JSP에서 그대로 읽을 수 있습니다. 결과적으로 “request scope의 수명을 확장하는 효과”가 발생합니다.

실습에서는 리다이렉트 시 `request scope` 는 `null` 로 나타나고, `session/application` 에 저장한 값은 유지됨을 확인했습니다. 이는 리다이렉트가 재요청을 발생시키되, 세션/애플리케이션 스코프는 요청을 넘어 유지되기 때문입니다.

리다이렉트의 대표 용도는 메인 페이지 이동, 로그인 페이지로 유도 같은 “페이지 전환”에 가깝고, 포워드는 요청 처리 결과를 request에 담아 JSP에서 즉시 렌더링해야 하는 경우에 적합합니다.

항목	리다이렉트(Redirect)	포워드(Forward)
URL 변화	변경됩니다.	변경되지 않습니다.
요청 횟수	요청이 2회 발생합니다.	요청이 1회만 유지됩니다.
request scope 전달	전달되지 않습니다.	전달됩니다.
주요 용도	메인/로그인 페이지 이동 등 전환 중심입니다.	결과 데이터를 JSP로 보여주는 렌더링 중심입니다.

핵심 키워드

#sendRedirect

#RequestDispatcher

#forward

#request scope

#URL 변경

4. HTTP와 세션 관리(세션, 쿠키)

HTTP는 요청-응답 후 연결이 끊어지는 **Connectionless** 특성이 있어 상태 공유가 불가하며, 이를 보완하기 위해 서버 저장소(세션) 또는 클라이언트 저장소(쿠키)로 상태를 유지합니다.

HTTP 프로토콜은 요청과 응답이 끝나면 연결 통로가 끊어지는 **Connectionless** 특성을 가집니다. 이는 불특정 다수 사용자가 동시에 접속하는 웹 환경에서 연결을 계속 유지하면 서버가 연결을 관리하느라 감당할 수 없기 때문에 채택된 방식입니다.

이 특성 때문에 A 서버릿에서 수행한 작업 상태를 B 서버릿이 “본래는” 알 수 없습니다. 그러나 실제 웹 애플리케이션은 로그인 상태 유지처럼 이전 작업을 이후 페이지가 인식해야 하므로, “페이지(서블릿/JSP) 간 데이터를 공유하도록 만드는 작업”이 필요하며 이를 세션 관리라고 부릅니다.

세션 관리는 대표적으로 세션과 쿠키로 구현되며, 가장 큰 차이는 저장 위치입니다. 세션은 서버에 저장하고 쿠키는 클라이언트(브라우저)에 저장합니다.

구분	세션(Session)	쿠키(Cookie)
저장 위치	서버(통캣)입니다.	클라이언트(브라우저)입니다.
저장 데이터 타입	Object 저장이 가능해 자바 객체 전반이 가능합니다.	문자열(String)만 저장됩니다.
보안 관점	상대적으로 안전합니다.	노출/공유 가능성이 있어 보안 이슈가 큼니다.
대표 용도	로그인 상태 유지에 주로 사용됩니다.	광고/개인화(추적) 등에도 활용됩니다.
추가 이슈	서버 자원 사용량 관리가 필요합니다.	사용자가 쿠키를 비활성화하면 동작이 제한됩니다.

세션은 브라우저 단위로 유지되며, 같은 브라우저(예: 크롬 내 탭들)는 세션을 공유할 수 있지만, 브라우저 벤더가 다르면(크롬 vs 엣지) 공유되지 않습니다. 이는 각 브라우저가 가진 세션 식별자(JSESSIONID)가 다르기 때문입니다.

핵심 키워드

#HTTP Connectionless

#세션 관리

#JSESSIONID

#세션

#쿠키

5. 세션 구현 패턴과 로그아웃

세션은 `request.getSession()` 으로 접근하여 키-밸류로 저장하고, 사용하는 컴포넌트는 값 존재 여부로 로그인 상태를 검증한 뒤 필요 시 리다이렉트로 우회합니다.

세션은 서버 측 저장소이며 키-밸류 구조로 데이터를 저장합니다. 브라우저가 요청을 보낼 때 보이지 않게 JSESSIONID가 함께 전달되며, 서버는 이 ID에 대응되는 세션 저장소를 찾아 재사용합니다.

세션 접근은 `request.getSession()` 또는 `request.getSession(true)` 로 하면 “없으면 생성, 있으면 반환”이고, `request.getSession(false)` 는 “없으면 null”을 반환합니다. 저장을 해야 하는 로그인 처리 같은 경우에는 세션이 반드시 필요하므로 `false` 는 부적합합니다.

세션 저장과 조회는 아래 패턴으로 정리됩니다.

- 세션에 데이터를 저장하는 컴포넌트는 세션을 생성/획득한 뒤 `setAttribute` 로 저장합니다.
- 세션 데이터를 사용하는 컴포넌트는 동일하게 세션을 획득한 뒤 `getAttribute` 결과가 null인지로 상태를 판단합니다.
- null이라면 로그인하지 않았거나 세션이 만료된 상태이므로 로그인 화면으로 유도해야 하며, 이때 리다이렉트를 사용합니다.
- null이 아니면 정상 상태이므로 비즈니스 처리를 진행하고 필요한 데이터만 request scope로 옮겨 뷰 (JSP)에 포워드합니다.

로그인 실습에서는 로그인 폼(JSP) → 로그인 서블릿(세션 저장) → 로그인 성공 JSP(세션 조회) → 마이페이지 서블릿(세션 검증 후 request 저장) → 마이페이지 JSP(포워드로 렌더링) 흐름을 구성했습니다.

마이페이지는 “보여주고 끝나는 정보”로 가정했기 때문에 세션에 주소/이메일 같은 데이터를 계속 남기지 않고 request scope에 저장한 뒤 포워드로 렌더링했습니다. 리다이렉트는 request scope를 전달할 수 없으므로 이 지점에서는 포워드가 필수입니다.

로그아웃은 세션 데이터를 삭제하는 기능이며, 삭제 방식은 두 가지로 나뉩니다.

삭제 단위	메소드	의미	예시
세션 전체 삭제	<code>session.invalidate()</code>	세션 저장소 자체를 폐기합니다.	로그아웃에 적합합니다.
특정 키만 삭제	<code>session.removeAttribute("key")</code>	특정 엔트리만 삭제합니다.	장바구니만 비우기 등에 적합합니다.

특정 항목만 지워야 하는 상황에서 세션 전체를 invalidate하면 로그인 정보까지 사라지므로, 목적에 맞게 삭제 전략을 선택해야 합니다.

핵심 키워드

#getSession(false)

#setAttribute

#getAttribute

#invalidate

#removeAttribute

6. 쿠키 생성, 조회, 삭제와 저장 위치

쿠키는 문자열만 저장 가능하며 `response.addCookie` 로 클라이언트에 심고, 이후 요청에서 `request.getCookies()` 로 배열 형태로 전달받아 키로 검색해 사용합니다.

쿠키는 `Cookie(String name, String value)` 로 생성하며, value가 문자열로 제한된다는 점이 세션과의 중요한 차이입니다. 클라이언트에 저장되므로 서버가 아닌 브라우저에서 다루는 데이터라는 전제를 반드시 유지해야 합니다.

쿠키는 응답을 통해 클라이언트에 전달되어야 하므로, `response.addCookie(cookie)` 가 필수입니다. 또한 쿠키는 브라우저 메모리에 저장될 수도 있고, 물리 파일로 저장될 수도 있는데 이는 만료 시간 설정으로 결정됩니다.

만료 시간은 `cookie.setMaxAge(int seconds)` 로 설정하며, 값의 의미는 다음과 같습니다.

setMaxAge 값	저장/동작	의미
양수	파일 저장(지속 쿠키)	브라우저를 종료해도 지정 시간 동안 유지됩니다.
음수	브라우저 메모리(세션 쿠키)	브라우저 종료 시 삭제됩니다.
0	삭제	즉시 삭제 요청에 해당합니다.

쿠키 조회는 요청에 포함되어 전달되므로 `request.getCookies()` 로 얻습니다. 리턴 타입이 쿠키 배열인 이유는 도메인당 여러 개의 쿠키를 가질 수 있기 때문이며, 수업에서는 도메인당 최대 3000개, 쿠키 1개 크기 제한 4KB를 함께 확인했습니다.

원하는 쿠키를 찾기 위해서는 배열을 순회하며 `cookie.getName()` 으로 키를 비교한 뒤 `cookie.getValue()` 를 사용합니다. 삭제도 동일하게 “찾은 쿠키에 `setMaxAge(0)` 적용 후 `response.addCookie(...)` 로 다시 응답”해야 클라이언트 측에서 실제로 삭제가 반영됩니다.

브라우저에서 쿠키는 개발자도구의 Application(또는 Storage) 영역에서 확인/삭제할 수 있으며, 사용자가 브라우저 설정으로 쿠키 자체를 비활성화할 수 있다는 점이 실무에서 큰 리스크가 됩니다. 또한 실무에서는 세션과 쿠키를 함께 사용하는 형태가 일반적입니다.

핵심 키워드

#Cookie

#addCookie

#getCookies

#setMaxAge

#4KB 제한

7. JSP 실행 단계와 JSP 구성 요소

JSP는 변환→컴파일→실행의 3단계로 동작하며, HTML과 JSP 태그(디렉티브/스크립틀릿/표현식), EL, JSTL을 조합해 뷰를 구성합니다.

JSP는 확장자가 `.jsp` 이며 실행 시 내부적으로 3단계를 거칩니다. 즉 JSP가 서블릿 소스(`.java`)로 변환되고, 컴파일되어 클래스가 된 뒤 실행되며, 최종적으로 HTML을 생성해 브라우저로 전달합니다.

JSP 파일 내부는 크게 HTML과 JSP 관련 문법 요소들로 구성됩니다. JSP 태그는 다음과 같이 정리됩니다.

분류	문법 예	명칭	대응 개념
디렉티브	<code><%@ page ... %></code>	페이지 디렉티브	설정/지시 역할입니다.
디렉티브	<code><%@ include ... %></code>	인클루드 디렉티브	포함 지시 역할입니다.
디렉티브	<code><%@ taglib ... %></code>	태그라이브러리 디렉티브	JSTL 등 태그 사용 선언입니다.
선언	<code><%! ... %></code>	디클레이션 태그	인스턴스 변수/메소드 성격입니다.
스크립틀릿	<code><% ... %></code>	스크립틀릿 태그	서비스 메소드(doGet 유사) 영역 성격입니다.
표현식	<code><%= ... %></code>	익스프레션 태그	출력(응답 HTML에 값 삽입) 성격입니다.

수업에서는 선언 태그(`<%! ... %>`)는 인스턴스 변수 기반이라 스레드 안전성과도 연결되며, 실무적으로는 사용할 일이 적다는 점을 함께 언급했습니다. 반면 `<% ... %>` 와 `<%= ... %>` 는 실습에서 request/session attribute 값을 가져오고 출력하는 데 사용했습니다.

추가로 JSP에서는 EL(Expression Language) 표기법인 `${ ... }` 를 사용할 수 있으며, JSTL(JavaServer Pages Standard Tag Library)은 반복/조건 같은 로직을 커스텀 태그 형태로 제공하는 라이브러리입니다. 예시로 조건 처리 태그(`c:if`) 같은 형태가 소개되었습니다.

핵심 키워드

#JSP 변환

#디렉티브

#스크립틀릿

#익스프레션

#JSTL

8. 모델1 vs 모델2(MVC) 아키텍처

모델1은 JSP가 로직과 화면을 모두 담당해 초기 개발은 빠르지만 유지보수가 어렵고, 모델2는 서블릿(컨트롤러)과 JSP(뷰)를 분리해 MVC로 확장 가능한 표준 구조입니다.

모델1 아키텍처는 브라우저가 JSP를 직접 요청하고, JSP가 서비스/DAO/DB 연동과 화면 렌더링을 모두 처리하는 구조입니다. 초기에 빠르게 개발할 수 있으나, 화면 코드와 자바 코드가 한 파일에 뒤섞여 규모가 커질수록 변경 영향 분석과 유지보수가 매우 어려워집니다.

모델2 아키텍처는 브라우저가 서블릿(컨트롤러)을 요청하고, 서블릿이 모델(서비스/DAO/DB)을 제어한 뒤 JSP(뷰)로 요청 위임하여 렌더링하게 합니다. 이때 서블릿이 JSP를 선택하는 과정이 곧 요청 위임(포워드/리다이렉트)이며, 구조적으로 책임이 분리되어 관리가 용이합니다.

모델2의 중요한 규칙은 “브라우저는 JSP를 직접 요청하지 않는다”는 점입니다. JSP로 직접 접근이 기술적으로 가능하더라도, 모델2 구조에서는 컨트롤러를 통해서만 뷰로 이동해야 아키텍처가 유지됩니다.

모델2는 MVC 패턴으로도 표현되며, 데이터 영역을 Model, 화면을 View(JSP), 양쪽을 제어하며 흐름을 결정하는 서블릿을 Controller로 정의합니다. 이 구조는 스프링 프레임워크의 기반이므로 반드시 기억해야 합니다.

핵심 키워드

#모델1

#모델2

#MVC

#컨트롤러

#요청 위임